Correction et prolongements du sujet 8

Les prolongements n'étaient évidemment pas exigés à l'examen

Exercice 1

```python
def recherche(caractere: str, mot: str) -> int:
    """
    renvoie le nombre d'occurences de 'caractere' dans la chaine éventuellement vide
'mot'.
    caractere, mot: chaînes (str)
    """
    nb_oc = 0
    for c in mot:
        if c == caractere:
            nb_oc += 1
    return nb_oc
```

```python
recherche('e', 'sciences')
```

```
2
```

```python
recherche('i', 'mississippi')
```

```
4
```

```python
recherche('a', 'mississippi')
```

```
0
```

Versions récursives (prolongement)

Version utilisant le "slicing"

```python
def recherche_f1(c, m, acc=0):
    """
    renvoie le nombre d'occurences de c dans la chaine éventuellement vide m.
    c, m: chaînes (str)
    acc: accumulateur (int), contient le nombre d'occurences
    """
    if m == '':
        return acc
    else:
        if c == m[0]:
            return recherche_f1(c, m[1:], acc + 1)
        else:
            return recherche_f1(c, m[1:], acc)
```

```python
recherche_f1('e', 'sciences')
```

```
2
```

```python
recherche_f1('i', 'mississippi')
```

```
4
```

```python
recherche_f1('a', 'mississippi')
```

```
0
```

Version sans slicing

```python
def recherche_f2(c, m, i=0, acc=0):
    """
    renvoie le nombre d'occurences de c dans la chaine éventuellement vide m.
    c, m: chaînes (str).
    acc: accumulateur (int), contient le nombre d'occurences;
    i: index (int) d'un caractère dans m.
    """
    if i >= len(m):
        return acc
    else:
        if c == m[i]:
            return recherche_f2(c, m, i+1, acc + 1)
        else:
            return recherche_f2(c, m, i+1, acc)
```

```python
recherche_f2('e', 'sciences')
```

```
2
```

```
recherche_f2('i', 'mississippi')
```

```
4
```

```
recherche_f2('a', 'mississippi')
```

```
0
```

Qu'en est-il des temps d'exécution ?

```
import string
import random

# construisons une chaine "aléatoire" de 1000 caractères
ch = ''.join([random.choice(string.ascii_letters) for _ in range(1000)])
```

```
%timeit recherche('a', ch)
```

```
29.8 µs ± 707 ns per loop (mean ± std. dev. of 7 runs, 10000 loops each)
```

```
%timeit recherche_f1('a', ch)
```

```
277 µs ± 8.15 µs per loop (mean ± std. dev. of 7 runs, 1000 loops each)
```

```
%timeit recherche_f2('a', ch)
```

```
221 µs ± 4.67 µs per loop (mean ± std. dev. of 7 runs, 1000 loops each)
```

La version itérative est plus rapide d'un facteur 10 environ.

Exercice 2 (modifié)

```python
Pieces = [100, 50, 20, 10, 5, 2, 1]

def rendu_glouton(arendre, solution, i=0):
    if arendre == 0:
        return solution
    p = Pieces[i]
    if p <= arendre:
        solution.append(p)
        return rendu_glouton(arendre - p, solution, i)
    else:
        return rendu_glouton(arendre, solution, i+1)
```

```python
rendu_glouton(68, [], 0)
```

```
[50, 10, 5, 2, 1]
```

```python
rendu_glouton(291, [], 0)
```

```
[100, 100, 50, 20, 20, 1]
```

L'exercice en version originale (prolongement)

La proposition erronée du sujet renfermait une erreur subtile (en dehors du problème d'indentation ou du passage de paramètre lors du 2ème `if`).

```python
Pieces = [100, 50, 20, 10, 5, 2, 1]

def rendu_glouton_i(arendre, solution=[], i=0):
    if arendre == 0:
        return solution
    p = Pieces[i]
    if p <= arendre:
        solution.append(p)
        return rendu_glouton(arendre - p, solution, i)
    else:
        return rendu_glouton(arendre, solution, i+1)
```

```python
rendu_glouton_i(68, [], 0)
```

```
[50, 10, 5, 2, 1]
```

```python
rendu_glouton_i(291, [], 0)
```

```
[100, 100, 50, 20, 20, 1]
```

Le résultat semble correct. Mais essayons d'exploiter les paramètres par défaut...

```
rendu_glouton_i(291)
```

```
[100, 100, 50, 20, 20, 1]
```

```
rendu_glouton_i(291)
```

```
[100, 100, 50, 20, 20, 1, 100, 100, 50, 20, 20, 1]
```

... ce qui est manifestement faux ! Le problème vient du paramètre par défaut `solution=[]`. La documentation python précise:

Les valeurs par défaut des paramètres sont évaluées de la gauche vers la droite quand la définition de la fonction est exécutée. Cela signifie que l'expression est évaluée une fois, lorsque la fonction est définie, et que c'est la même valeur « pré-calculée » qui est utilisée à chaque appel. C'est particulièrement important à comprendre lorsqu'un paramètre par défaut est un objet mutable, tel qu'une liste ou un dictionnaire : si la fonction modifie l'objet (par exemple en ajoutant un élément à une liste), la valeur par défaut est modifiée.

En d'autres termes c'est le même objet *liste* qui est appelé à chaque exécution de la fonction.

D'une manière générale, il faut éviter de mettre des structures mutables comme valeur par défaut

Une solution issue de l'exploitation de la documentation

```
Pieces = [100, 50, 20, 10, 5, 2, 1]

def rendu_glouton_ii(arendre, solution=None, i=0):
    if solution is None:
        solution = []
    if arendre == 0:
        return solution
    p = Pieces[i]
    if p <= arendre:
        solution.append(p)
        return rendu_glouton(arendre - p, solution, i)
    else:
        return rendu_glouton(arendre, solution, i+1)
```

```
rendu_glouton_ii(291)
```

```
[100, 100, 50, 20, 20, 1]
```

```
rendu_glouton_ii(291)
```

```
[100, 100, 50, 20, 20, 1]
```

```
rendu_glouton_ii(68)
```

```
[50, 10, 5, 2, 1]
```

```

```